Project 3 仓库说明（中文版）

AIAA 3201 Project 3: Video Object Removal & Inpainting

2026 年 4 月 11 日

1 文档目的

这份文档是当前仓库的中文版使用说明，主要回答三类问题：

- 仓库里每个文件和目录是做什么用的；
- 目前哪些部分已经实现，哪些部分只是为后续实验预留接口；
- 因为 GPU 资源在服务器上，接下来你们应该按什么顺序把这套代码迁移到服务器并真正跑起来。

如果你们后面忘了某个文件的作用，优先先看这份文档，而不是直接猜。

2 项目当前总体结构

当前仓库围绕下面这条主线组织：

动态目标掩码生成 -> 掩码优化 -> 视频补全 -> 定量评测 -> 可视化 -> 报告写作

整体上已经具备：

- Part 1 baseline 的代码骨架与命令入口；
- Part 2 的外部模型接入接口；
- Part 3 的 failure case 工作区准备工具；
- 指标计算、可视化、报告和展示材料的模板。

3 顶层文件说明

3.1 `README.md`

这是仓库的主入口文档。

主要用途：

- 解释整个项目的目标；
- 展示目录结构；
- 给出环境安装方式；
- 给出 Part 1 运行方式；
- 给出评测命令；
- 给出 Part 2 / Part 3 的推荐工作流。

任何第一次打开仓库的人，应该先读它。

3.2 `requirements.txt`

这是 Python 依赖列表。

里面当前列出的核心包包括：

- `opencv-python`：读写视频、光流、掩码处理、图像修复；
- `ultralytics`：YOLOv8-seg；
- `PyYAML`：读取配置文件；
- `scikit-image`：SSIM；
- `numpy / scipy / matplotlib`：基础数值和可视化支持。

3.3 `.gitignore`

这个文件是为了避免把大文件误提交到仓库里。

它会忽略：

- 原始数据视频；
- 抽帧结果；

- 实验输出；
- 指标结果；
- 虚拟环境和缓存文件。

因为视频项目的中间产物非常大，这个文件很重要。

4 配置文件说明

4.1 configs/project3.yaml

这是默认实验配置文件，也是后续最常修改的文件之一。

它控制：

- YOLO 模型名称；
- 要保留的目标类别；
- 光流动态判断阈值；
- 掩码膨胀参数；
- 时间窗口大小；
- Part 2 中 `SAM 2` 和 `ProPainter` 的外部路径；
- Part 3 的说明性参数。

可以把它理解成实验控制面板。

5 代码入口说明

5.1 scripts/project3.py

这是你们之后最常直接运行的入口脚本。

它封装了命令行接口，目前支持：

- `part1`：运行 Part 1 baseline；
- `eval-mask`：计算 mask 指标；
- `eval-video`：计算恢复结果指标；
- `figures`：生成对比图；

- `part2-prepare`：建立 Part 2 实验工作区；
- `part3-prepare`：建立 Part 3 failure case 工作区。

这个脚本存在的意义是让你们不需要手动处理 `PYTHONPATH`。

6 src/project3/ 代码逐文件说明

6.1 config.py

负责读取配置文件。

除了标准 YAML 读取外，它还额外提供了一个轻量 fallback 解析器。这样做的目的是：即使本地环境暂时没有安装 `PyYAML`，仓库默认配置也仍然能被读取。

6.2 io_utils.py

这是一些小型文件工具函数。

它负责：

- 自动创建目录；
- 列出图像文件；
- 写 JSON 摘要文件。

它本身不做视觉算法，只是供其他模块复用。

6.3 video_utils.py

这是视频层面的底层工具函数。

它负责：

- 读取视频并拆成帧；
- 把帧写回成视频；
- 把一组帧保存成图片序列。

所有 Part 1 的读写流程都依赖这个模块。

6.4 part1.py

这是当前最核心的实现文件，也是现在真正有算法内容的主体。

它实现了 Part 1 baseline 的完整流程：

1. 读取输入视频；
2. 对每帧运行 YOLOv8-seg；
3. 保留指定类别的候选目标；
4. 在候选目标内部用 Lucas-Kanade sparse optical flow 判断该目标是否真的是动态目标；
5. 用膨胀和连通域清理优化掩码；
6. 先用邻近帧的时间中值去借背景；
7. 若仍有空洞，再使用 `cv2.inpaint` 作为补充；
8. 保存输入帧、mask、恢复帧、恢复视频和摘要 JSON。

简单说，`part1.py` 就是你们当前第一版“能跑通、能交差、还能做对比”的方法。

6.5 eval_metrics.py

这个模块负责评测。

它支持两类指标：

- mask 指标：`JM`、`JR`、precision；
- 恢复质量指标：`PSNR`、`SSIM`。

这个文件主要在有 GT 数据时使用，比如 DAVIS 或你们自己整理出的带真值子集。

6.6 visualization.py

这个模块负责把实验结果整理成报告和 PPT 好用的对比图。

当前生成的图会展示：

- original frame；
- predicted mask；
- restored frame。

它的意义不是“让代码更好看”，而是让你们写报告时直接有图可用。

6.7 part2.py

这个文件不是在本仓库里重写 SAM 2 或 ProPainter，而是做一个“适配层”。

它负责：

- 创建 Part 2 的标准输出目录；
- 写入 metadata；
- 生成推荐的调用命令模板。

这样做的目的，是让你们把强模型放在外部仓库里维护，同时保证当前主仓库的组织是统一的。

6.8 part3.py

这个模块是给 Part 3 的 failure case 实验准备工作区的。

它负责为某个失败案例创建：

- 关键帧目录；
- mask 目录；
- 编辑后关键帧目录；
- 比较结果目录；
- 记录这个 failure case 目标的说明 JSON。

它的作用是帮助你们把 Part 3 控制在“小范围、有证据、便于展示”的状态，而不是一开始就把 diffusion 扩展做成一个大工程。

6.9 cli.py

这是命令行接口的总调度模块。

所有在 scripts/project3.py 里调用的命令，真正都会转到这里来执行。

6.10 __init__.py

这是标准的 Python 包标记文件，让 project3 目录可以被 Python 识别成模块。

7 文档目录说明

7.1 docs/submission_checklist.md

最终提交前的操作清单。

主要用来确认：

- GitHub 仓库是否整理好；
- 报告是否完整；
- arXiv 是否上传；
- `videos.zip` 是否包含 mandatory 数据集结果；
- 图表、链接、引用是否齐全。

7.2 docs/presentation_outline.md

这是 8 分钟 progress presentation 的讲稿结构建议。

它的目标不是帮你们写逐字稿，而是提醒你们：在 Week 11/12 的汇报里，最少要覆盖哪些内容。

7.3 docs/pipeline_notes.md

这是一个更偏执行层面的流程说明。

相比 README，它更强调：

- Part 1、Part 2、Part 3 应该如何衔接；
- 什么时候启动 Part 2；
- 什么时候才适合开始 Part 3。

7.4 docs/repo_guide.md

英文版仓库说明。

7.5 docs/repo_guide_zh.tex

也就是你现在在看的这份中文版 LaTeX 文档。

8 报告相关文件说明

8.1 report/outline.md

这是一个自然语言版的论文结构草稿。

适合你们在真正写论文前，先决定每节写什么。

8.2 report/main.tex

这是论文主文档的 LaTeX 骨架。

当前它只是一个轻量起步模板，不是正式的 CVPR 官方模板。

真正交作业时，你们应该把内容迁移到课程要求的 CVPR 模板中。

8.3 report/references.bib

这是参考文献占位文件。

后续必须把 project PDF 里列出的 references 全部补进去，并确保正文里都被引用到。

9 数据和输出目录说明

9.1 data/raw/

放原始视频和 GT 数据。

建议使用：

- `data/raw/wild/`
- `data/raw/bmx/`
- `data/raw/tennis/`
- `data/raw/davis/`

9.2 data/frames/

如果你们想额外做手动抽帧或中间处理，可以把结果放在这里。

9.3 outputs/part1/

存 Part 1 baseline 的输出。

每个实验通常应包含：

- input frames；
- masks；
- restored frames；
- restored video；
- summary JSON；
- comparison figure。

9.4 `outputs/part2/`

存 Part 2 的输出，比如：

- refined masks；
- ProPainter 输入；
- ProPainter 输出；
- metadata 和推荐命令。

9.5 `outputs/part3/`

存 failure case 和关键帧编辑实验的结果。

9.6 `metrics/`

存导出的指标 JSON。

10 现在已经完成了什么

目前已经完成的部分：

- 仓库结构；
- baseline Part 1 代码；
- 评测代码；
- 可视化代码；
- Part 2 / Part 3 工作区准备工具；

- 报告和展示用文档骨架。

目前还没有在本地真正跑起来的部分：

- YOLOv8-seg 实际推理，因为本地尚未安装相关依赖；
- SAM 2 / ProPainter，因为它们要在服务器环境和 GPU 上运行；
- 完整实验，因为数据还没有放进仓库目录。

11 为什么现在不要继续在本地硬跑

因为你们的 GPU 资源在服务器上，所以本地当前阶段更适合做：

- 仓库整理；
- 文档准备；
- 代码结构搭建；
- 路线设计；
- 报告模板和参考文献准备。

真正的模型推理、长视频处理、SOTA 模型接入，都应该放到服务器上做。

12 接下来在服务器上的推荐操作顺序

Step 1：同步当前仓库到服务器

把这个仓库 clone 到服务器工作目录，或者直接上传整个项目文件夹。

Step 2：创建环境并安装依赖

推荐命令：

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

如果服务器已有 CUDA 版本的 PyTorch，请优先保证 `ultralytics`、SAM 2 和 ProPainter 与那套环境兼容。

Step 3：放置 mandatory 数据集

把视频放到：

```
data/raw/wild/
data/raw/bmx/
data/raw/tennis/
```

如果你们想拿更强的量化结果，也把 DAVIS 放好。

Step 4：先跑 Part 1 baseline

推荐先跑 wild video，再跑 bmx 和 tennis。

示例命令：

```
python3 scripts/project3.py part1 \
  --config configs/project3.yaml \
  --video data/raw/wild/corridor.mp4 \
  --experiment wild_corridor
```

先做 Part 1 的原因：

- 它最快形成第一个完整闭环；
- 它能尽快产出 progress presentation 可用结果；
- 它给 Part 2 提供清晰的对比基线。

Step 5：人工检查输出

重点看四件事：

- mask 是否只覆盖动态目标；
- 细结构是否被漏掉；
- 背景是否明显模糊；
- 是否有时间闪烁。

这个步骤不能省，因为报告里需要 qualitative analysis，不是只贴指标。

Step 6：准备 Part 2

先运行：

```
python3 scripts/project3.py part2-prepare \
  --config configs/project3.yaml \
  --video data/raw/wild/corridor.mp4 \
  --experiment wild_corridor
```

然后再做下面几件事：

- clone 或安装 `SAM 2`；
- clone 或安装 `ProPainter`；
- 在 `configs/project3.yaml` 中写入服务器上的真实路径和 checkpoint；
- 根据输出的推荐命令模板真正执行模型。

Step 7：计算指标

一旦有 GT 支持的数据，就运行：

- `eval-mask`；
- `eval-video`。

目标是尽快得到：

- `JM / JR`；
- `PSNR / SSIM`。

Step 8：最后再启动 Part 3

不要过早做 diffusion 扩展。

应当满足以下条件后再开始：

- Part 1 已稳定；
- Part 2 已稳定；
- 你们已经能明确指出 ProPainter 的真实 failure case。

这样才能保证 Part 3 是一个“有针对性的扩展”，而不是变成无底洞。

13 两人小组的推荐分工

同学 A

- 搭服务器环境；
- 跑 Part 1 baseline；
- 检查 wild / bmx / tennis 的输出效果；
- 整理 baseline 的可视化图。

同学 B

- 整理数据集目录；
- 准备报告模板与 references；
- 阅读 SAM 2 / ProPainter 的安装和运行要求；
- 准备 progress presentation 内容。

等 Part 1 稳定后，两人再一起合流做 Part 2。

14 最后的实用提醒

- 不要在 Part 1 和 Part 2 还没稳定前就过度投入 Part 3；
- 不要把定性图拖到最后再整理；
- 不要拖延 references 管理，因为课程明确要求引用给定文献；
- 不要默认服务器路径和本地路径一致，迁移后要检查 `configs/project3.yaml`。

15 结论

现在这个仓库的定位不是“已经完成所有实验”，而是“已经把项目工程骨架、baseline 逻辑、评测接口和文档模板都搭好”。你们下一阶段的重点不是继续本地开发，而是把这套结构迁移到服务器，先跑通 Part 1，再逐步接入 Part 2 和 Part 3。